

Cours n° 08

CODAGE DES NOMBRES

I NUMÉRATION

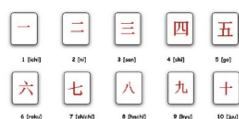
Comment comptent les humains? Cela dépend de leur civilisation.

Les nombres entiers 1,2,3,... sont "absolus". Trois chèvres, c'est trois chèvres où que l'on soit dans le monde. Mais la manière de représenter ces nombres est très, très diverse.

Les origines du calcul remontent à la Préhistoire où l'on comptait avec des cailloux et ses doigts. Le mot calcul vient d'ailleurs du latin calculi signifiant caillou.

Chaque civilisation a développé son propre système de numération et ses méthodes pour compter et calculer.

Un système de numération peut-être vu comme un premier codage des nombres.



Chiffres Japonais



Chiffres Babyloniens

La première notion importante est la base : on compte n objets, on les place dans un paquet, on en recompte n on les place dans un autre paquet, et quand on a n paquets, on les met dans un grand paquet.

Les bases les plus courantes sont :

- La base 5 : Les cinq doigts de la main. Utilisée comme sous-base par les Romains ou les Mayas.
- La base 10 : Les doigts des deux mains. Notre système actuel.
- La base 12 : Le nombre de phalanges des doigts, le pouce servant à compter. "Utilisée" encore en Inde, au Pakistan. Le fait que certains aliments soient comptés par douzaine (les oeufs) sont des restes de cette base. Les Sumériens partageaient le jour en 12 heures.
- La base 20 : Le nombre total de doigts, mains et pieds. Utilisé par les Mayas, Aztèques et les Celtes. Pour eux 40 était désigné par "deux vingts" et 80 par "quatre vingts".
- La base 60 : Une combinaison des bases 12 et 5. Une main en base 12, et les doigts de l'autre main comptaient les douzaines. Les Babyloniens utilisaient cette base. Elle est en fait très pratique : 60 est divisible par 2,3,4,5,6,10... . Le décompte des secondes et minutes nous vient de là.

Il est intéressant de voir que notre langage et nos habitudes, *a fortiori* notre culture, sont une agglomération de cultures plus anciennes.

Une des percées scientifiques de l'humanité est la numération de position. Il y a en gros deux systèmes de numération dans l'humanité :

- La numération additive, utilisée par exemple par les Romains : X pour dix, et XX pour vingt.
- La numération de position : la position du chiffre dans l'écriture est déterminante. On écrit (en base 10) 12 pour signifier une dizaine et deux unités.

Le système positionnel permet de faire des calculs plus simplement. Compter rapidement a permis à la Science d'accélérer son expansion.

DÉFINITION 1

Un entier naturel X s'écrit dans la base B sous la forme suivante :

$$X = a_n B^n + a_{n-1} B^{n-1} + \dots + a_0 B^0 = \sum_{i=0}^n a_i B^i = (a_n a_{n-1} \dots a_1 a_0)_B$$

Par exemple : $(2301)_4 = 2 \times 4^3 + 3 \times 4^2 + 0 \times 4 + 1 = 180$.

Les bases les plus utilisées sont :

- la base 10, dont les chiffres sont 0,1,2,3,4,5,6,7,8,9
- la base 2 (binaire) la base 10, dont les chiffres sont 0,1
- la base 16 (hexadécimale), dont les chiffres de 0 à 15 sont notés 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

MÉTHODE

Pour passer d'une base B à la base 10 : il suffit d'appliquer la définition.

Pour passer d'une base 10 à la base B : on procède par divisions euclidiennes successives par B .

On passe facilement de la forme binaire à la forme hexadécimale en regroupant les chiffres binaires par 4.

EXEMPLE 1. Écrire 145 en base 2 puis 16. Faire de même avec 1984.

II NUMÉRISATION

Numériser l'information consiste à la quantifier, c'est à dire à lui attribuer une valeur, prise dans un ensemble fini de valeurs. On note donc toute la difficulté et les limites de l'information numérique.

Le problème, c'est problème est qu'en machine on ne dispose que de deux états : du courant ou pas. C'est une information binaire, représentée souvent par 0 et 1. C'est une information :

- Imparfaite : discrétisation du réel
- Équivoque : on doit connaître le code pour le rendre intelligible
- Robuste : on distingue aisément 1 de 0
- Adaptable : nombre de supports peuvent l'accueillir
- Universelle

Il faut donc créer un système lié à cet état, la numération binaire s'impose. C'est assez simple à mettre en place pour des entiers positifs, un peu plus technique pour des entiers relatifs, très problématique pour les nombres réels, et virtuellement impossible pour des caractères, sons et images, où l'on s'adapte en "découpant".

Le bit est un chiffre binaire, c'est-à-dire 0 ou 1. Il est l'unité de mesure en informatique, celle désignant la quantité élémentaire d'information représentée par un chiffre du système binaire.

L'octet est en informatique une unité de mesure informatique, elle correspond à 8 bits. Attention, en anglais un octet se dit Byte, à ne pas confondre avec le Bit!

III LES ENTIERS

1 LES ENTIERS NATURELS

L'écriture en base 2 donne tout de suite un moyen de coder les entiers naturels, en affectant chaque chiffre à un bit.

Le problème est que la mémoire d'une machine n'as pas de notion de début et fin d'une séquence. On doit donc préciser dans une convention le nombre de bits que l'on veut utiliser pour coder les entiers.

- Sur 1 octet on peut coder : entiers
- Sur 16 bits on peut coder : entiers
- Sur 32 bits on peut coder : entiers
- Sur 64 bits on peut coder : entiers

Les processeurs ont une capacité en bits, qui est simplement la taille des entiers avec lesquels il peut calculer (après il découpe et s'y retrouve, mais c'est plus long).

On essaye alors de toujours représenter les bits de la norme, même les 0 à gauche. Sur 8 bits on écrira : 0010 1001 et non 101001.

L'addition est simple :

$$0100\ 0101 + 0100\ 0001 =$$

Il faut être conscient des problèmes de dépassement éventuels :

$$1110\ 0101 + 1100\ 0001 =$$

2 LES ENTIERS SIGNÉS

Une première idée est d'utiliser le premier bit pour le signe et de coder ensuite la valeur absolue, sur 4 bits par exemple :

- 1010 représente -2
- 0010 représente 2

Avantage : C'est simple, on connaît directement le signe d'un nombre.

Désavantages :

- On code $+0$ et -0
- Un bit n'a pas la même signification que les autres
- L'addition est problématique :

$$0000\ 0011 + 1000\ 0100 =$$

On utilise une méthode appelée **le complément à 2**. Sur N bits.

- 1 bit détermine le signe, le bit de poids fort à gauche
 - les autres bits (7, 15, 31 et 63 en pratique) déterminent la valeur :
 - ◇ pour un nombre positif, on utilise le codage binaire naturel
 - ◇ pour un nombre négatif, on utilise le complément à 2, on code en binaire naturel le nombre $2^N - |a|$
- ainsi on représente sur N bits les entiers de -2^{N-1} à $2^{N-1} - 1$.

Sur 16 bits par exemple on peut représenter 65536 nombres, les entiers entre -32768 et 32767, de la manière suivante :

- Un entier relatif positif x est représenté par le codage binaire l'entier naturel x
- Un entier relatif négatif x est représenté par le codage binaire l'entier naturel $x + 2^{16}$

Par exemple $-1 = 1111\ 1111\ 1111\ 1111$ et $-12 = 1111\ 1111\ 1111\ 0100$

De cette manière l'addition fonctionne, un bit dépassant à gauche représentant un changement de signe : En effet :

$$1111\ 1111 + 0000\ 0010 =$$

Mais il y a toujours des soucis :

$$1000\ 0001 + 1000\ 0001 =$$

3 ENTIERS MULTI-PRÉCISION EN PYTHON

On vient de voir que suivant le nombre d'octets utilisés pour représenter un entier, on a une limitation du nombre de valeurs représentables. Cela signifie que si l'on ne fait pas attention, le résultat d'une opération peut dépasser la capacité de représentation et une erreur de type *overflow* apparaîtra.

Il faut noter que le type `int` de Python est un type à représentation infinie. Il n'y a aucune limitation, Python modifiera le nombre de bits nécessaires au stockage à la volée. Mais ça peut ralentir énormément les algorithmes!

```
1 >>> entier = 45
2 >>> type(entier)
3 <class 'int'>
4 >>> entier.bit_length()
5 6
6 >>> entier=126874861321074574641412
7 >>> entier.bit_length()
8 77
```

La fonction `type` renvoie le type de la variable.

La méthode `bit_length()` renvoie le nombre de bits utilisés pour représenter l'entier.

IV LES RÉELS

1 ÉCRITURE DÉCIMALE ET BINAIRE DES RÉELS

Que signifie : 5,75? Il s'agit d'une notation pour : $5 \times 10^0 + 7 \times 10^{-1} + 5 \times 10^{-2}$.

Est ce qu'on peut écrire cela en binaire?

Oui, on sépare la partie entière et la partie décimale :

$7 = 4 + 1 = (101)_2$ et $0,75 = 0,5 + 0,25 = 2^{-1} + 2^{-2}$. Au final $5,75 = (101,11)_2$.

Inversement l'écriture décimale de $(10,1)_2 = 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} = 2,5$.

On se dit que c'est bon c'est réglé.

Peut-on représenter $1/3$ en base 10? Le développement sera infini. Cela sera le même problème en base 2.

On comprend vite qu'il est illusoire de vouloir coder les nombres réels pour eux même. Et tant bien même on y arrive, la question est : où place-t-on la virgule?

2 FLOTTANTS

Les flottants sont un sous ensemble fini des nombre décimaux, correspondant à ceux représentés (représentables?) en machine.

La méthode consiste à écrire ces nombres réels sous la forme dite en virgule flottante normalisée :

- $N = \pm a10^n$ où $a \in [1, 10[$ et $n \in \mathbb{Z}$ en base 10
- $N = \pm a2^n$ où $a \in [1, 2[$ et $n \in \mathbb{Z}$ en base 2.

où :

$\pm$ est le signe

a est la mantisse : un nombre entre 1 et 10 (ou 1 et 2 en base 2) exclus, avec donc un seul chiffre non nul avant la virgule

n l'exposant : un entier relatif.

Il faut néanmoins préciser le nombre de bits à utiliser pour chacun, et définir les règles de calcul. C'est le but de la norme **IEEE 754** qui est devenu un standard. Sur 64 bits :

- 1 bit pour le signe
- 11 bits pour l'exposant
- 52 bits pour la mantisse

Il y a de nombreuses exceptions (le zéro, des nombres dénormalisés) et un codage spéciale pour les infinis et les erreurs *Nan* (Not a Number) qui sont hors programme.

Les erreurs d'arrondis sont inévitables :

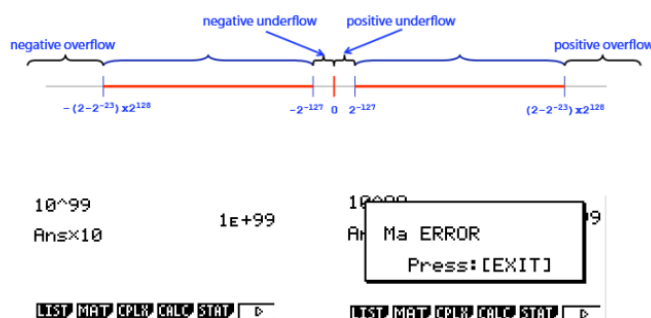
In [1] : $12 * (1/3 - 1/4) - 1$

Out [1] : $-2.220446049250313e-16$

In [2] : $(0.2 + 0.1) - 0.3$

Out [2] : $5.551115123125783e-17$

Les problèmes de dépassement sont multiples, et dépendent de l'environnement :



3 CALCULS EN FLOTTANTS

Les calculs effectués avec des flottants sont des calculs approchés

10.0^{16} et $10.0^{16} + 1$ c'est le même nombre en python (alors que 10^{16} et $10^{16} + 1$ non).

```
1 In [1]: a=10.0**16
2 In [2]: b=10.0**16+1
3 In [3]: a==b
4 Out [3]: True
```

La règle d'or : **Ne jamais faire de test d'égalité entre flottants.**

A $x==y$ on préférera $\text{abs}(x-y) \leq 10^{-6}$ en adaptant selon la précision voulue.

Une page interactive avec beaucoup de compléments sur le sujet : <https://mpechaud.fr/scripts/representationnombres/rep.html>.